# Jenkins Notes - Part 1

## 1. Introduction to Jenkins

### 1.1 Definition

Jenkins is an open-source automation server written primarily in Java. It is used to automate software development activities such as building, testing, integrating, and deploying applications.

Jenkins supports Continuous Integration and Continuous Delivery/Deployment (CI/CD) through pipelines and plugins.

### 1.2 Purpose - Why do we use Jenkins?

Jenkins is used to:

- Automate repetitive software development tasks.
- Integrate code changes from multiple developers.
- Execute automated tests.
- Build application packages and Docker images.
- Automate application and infrastructure deployments.
- Integrate with tools such as Git, GitHub, Maven, Docker, AWS, Terraform, and Kubernetes.
- Reduce manual effort and human errors.
- Provide build history, console logs, and pipeline status.

### 1.3 Working - How does Jenkins work?

1. A developer writes or modifies application code.
2. The developer pushes the code to a Git repository.
3. Jenkins starts a pipeline through a webhook, polling, schedule, or manual trigger.
4. Jenkins retrieves the source code.
5. Jenkins executes configured build and test commands.
6. If the required checks pass, Jenkins proceeds to packaging or deployment.
7. Jenkins records the execution result and provides logs.

### 1.4 Practical Example

Suppose a company develops a Java web application. Without Jenkins, developers may need to manually download the code, build the application, run tests, and deploy it. With Jenkins, the process can be automated:

```
Developer
    |
    v
GitHub Repository
    |
    v
Jenkins Pipeline
    |
    v
Build Application
    |
    v
Run Tests
    |
    v
Create Docker Image
    |
    v
Deploy to Kubernetes
```

Whenever a new code change is pushed, Jenkins can execute the configured pipeline automatically.

### 1.5 Interview Questions

#### Q1. What is Jenkins?

**Answer:** Jenkins is an open-source automation server used to automate software building, testing, and deployment. It is widely used to implement CI/CD pipelines.

#### Q2. Why is Jenkins used in DevOps?

**Answer:** Jenkins automates the software delivery process, integrates with development and deployment tools, reduces manual intervention, and helps deliver software faster and more consistently.

#### Q3. Is Jenkins a CI tool or a CD tool?

**Answer:** Jenkins supports both CI and CD. It can automate code integration, testing, packaging, and deployment through configurable pipelines.

#### Q4. Is Jenkins open-source?

**Answer:** Yes. Jenkins is an open-source automation server with a large plugin ecosystem.

#### Q5. What are the main features of Jenkins?

**Answer:** Its main features include pipeline automation, plugin integration, distributed builds, build history, source control integration, and automated deployment.

## 2. Continuous Integration (CI)

### 2.1 Definition

Continuous Integration is a software development practice in which developers frequently integrate their code changes into a shared repository. Each integration can trigger automated builds and tests to identify errors early.

### 2.2 Purpose - Why do we use CI?

- To identify code defects early.
- To detect integration problems quickly.
- To reduce conflicts between developers' changes.
- To maintain code quality.
- To provide fast feedback to developers.
- To reduce the risk of combining multiple changes at release time.

### 2.3 Working - How does CI work?

1. Developers write code independently.
2. They commit and push their changes to GitHub or another Git repository.
3. Jenkins detects the change or receives a pipeline trigger.
4. Jenkins checks out the latest code.
5. Jenkins compiles or builds the application.
6. Jenkins executes unit tests and other configured checks.
7. Jenkins reports success or failure.
8. If a build fails, the team investigates and fixes the issue before proceeding.

### 2.4 Practical Example

A team has five developers working on the same application. One developer modifies the login feature, while another updates the payment feature. When a developer pushes a change, Jenkins can automatically build the application and run tests. If the changes introduce a compilation error or cause a test to fail, the team receives early feedback.

```
Code Commit
    |
    v
GitHub
    |
    v
Jenkins Trigger
```

```
    |
    v
Checkout Code
    |
    v
Compile / Build
    |
    v
Automated Tests
    |
    v
Pass or Fail
```

### 2.5 Interview Questions

**Q1. What is Continuous Integration?**

**Answer:** Continuous Integration is the practice of frequently integrating code changes into a shared repository and validating them using automated builds and tests.

**Q2. What are the benefits of CI?**

**Answer:** CI helps identify defects early, improves collaboration, reduces integration risks, and provides faster feedback.

**Q3. How does Jenkins support CI?**

**Answer:** Jenkins integrates with source control systems and executes automated pipelines whenever configured triggers occur.

**Q4. What happens if a CI build fails?**

**Answer:** Jenkins marks the build as failed or unstable depending on the step and configuration, displays the console logs, and allows developers to investigate the failure.

**Q5. What is the difference between Continuous Integration and traditional integration?**

**Answer:** Traditional integration may combine changes infrequently, making defects difficult to isolate. CI integrates changes frequently and uses automated checks to detect problems earlier.

## 3. Continuous Delivery and Continuous Deployment (CD)

### 3.1 Definition

Continuous Delivery is a software development practice in which every change that passes the required validation is prepared for release. Production deployment may require manual approval.

Continuous Deployment is a practice in which every change that passes the required automated checks is automatically deployed to production.

Both practices extend the CI process by automating later stages of software delivery.

### 3.2 Purpose - Why do we use CD?

- To reduce deployment time.
- To standardize release procedures.
- To reduce manual deployment errors.
- To make releases more frequent and predictable.
- To improve rollback and release management processes.
- To deliver tested software more efficiently.

### 3.3 Working - How does CD work?

A typical delivery pipeline follows these stages:

1. Retrieve the source code.
2. Build the application.
3. Run automated tests.

4. Perform code quality and security checks.
5. Package the application or build a Docker image.
6. Publish the artifact or image to a repository.
7. Deploy to a test or staging environment.
8. Perform validation and approval where required.
9. Deploy to production according to the release strategy.

The exact stages depend on the project and its security requirements.

## 3.4 Practical Example

A company deploys a Java application using Jenkins, Docker, and Amazon EKS.

```
GitHub
   |
   v
Jenkins CI/CD Pipeline
   |
   v
Build & Test Application
   |
   v
Build Docker Image
   |
   v
Push Image to Amazon ECR
   |
   v
Deploy to Amazon EKS
   |
   v
Verify Application Health
```

In Continuous Delivery, production deployment might wait for an authorized person to approve the release.

In Continuous Deployment, a successfully validated change can be deployed automatically, provided all required policies and checks pass.

## 3.5 Interview Questions

### Q1. What is Continuous Delivery?

**Answer:** Continuous Delivery automates the process of building, testing, and preparing software for release, with production deployment commonly requiring approval.

### Q2. What is Continuous Deployment?

**Answer:** Continuous Deployment automatically releases changes to production after they pass the required automated checks.

### Q3. What is the difference between Continuous Delivery and Continuous Deployment?

**Answer:** Continuous Delivery keeps software ready for release, usually with a manual production approval step. Continuous Deployment automatically deploys validated changes to production.

### Q4. What is a deployment pipeline?

**Answer:** A deployment pipeline is an automated sequence of stages that moves software from source code through build, testing, packaging, and deployment.

### Q5. How can Jenkins help with rollback?

**Answer:** Jenkins can execute a configured rollback procedure, such as redeploying a previous Docker image or reverting to a previous application release. Rollback is not automatic unless the pipeline is designed to perform it.

# 4. Jenkins Architecture

## 4.1 Definition

Jenkins architecture describes how the Jenkins controller, agents, executors, and supporting components work together to execute automation tasks.

Jenkins supports both simple installations, where jobs run on the built-in node, and distributed installations, where jobs run on separate agents.

## 4.2 Purpose - Why do we need Jenkins architecture?

- To organize pipeline execution.
- To separate pipeline orchestration from build execution.
- To distribute workloads across multiple machines.
- To support different operating systems and build environments.
- To improve scalability and resource utilization.
- To isolate workloads when appropriate.

## 4.3 Working - How does Jenkins architecture work?

The main components are:

- **Jenkins Controller:** Manages Jenkins configuration, jobs, scheduling, and agents.
- **Jenkins Agent:** Executes tasks assigned by the controller.
- **Executor:** Represents a slot that can run a build or task on a node.
- **Jenkins Pipeline:** Defines the stages and steps to execute.
- **Plugins:** Add integrations and functionality, such as source control, cloud credentials, notifications, and build tools.

The controller coordinates pipeline execution. Depending on the pipeline and configuration, work runs on the controller's built-in node or on a separate agent.

## 4.4 Practical Example

Consider a company managing several projects:

- Java application builds run on a Linux agent.
- Windows-specific builds run on a Windows agent.
- Deployment tasks run on an appropriately configured agent with access to the required AWS resources.

A single controller can coordinate these different workloads.

- **Jenkins Controller:** Configuration, scheduling and orchestration
- **Linux Agent:** Build and test
- **Windows Agent:** Windows workloads

## 4.5 Interview Questions

### Q1. Explain Jenkins architecture.

**Answer:** Jenkins uses a controller-agent architecture for distributed execution. The controller manages configuration and schedules work, while agents execute assigned tasks. Executors provide the capacity to run tasks on each node.

### Q2. Can Jenkins run jobs without an agent?

**Answer:** Yes. Jobs can run on the controller's built-in node if configured to do so. However, dedicated agents are generally preferred for production workloads to reduce the controller's workload and improve isolation.

### Q3. What is distributed building in Jenkins?

**Answer:** Distributed building means executing jobs across multiple agents instead of relying on a single machine.

#### Q4. What is the role of plugins in Jenkins architecture?

**Answer:** Plugins extend Jenkins functionality and integrate it with tools and platforms such as GitHub, Docker, AWS, and Kubernetes.

#### Q5. How can Jenkins architecture be scaled?

**Answer:** Add appropriately configured agents, allocate executors based on available resources, and separate workloads according to operating system, tools, security, and capacity requirements.

## 5. Jenkins Controller

### 5.1 Definition

The Jenkins controller is the central service that manages the Jenkins environment. It provides the user interface, stores configuration, schedules work, and coordinates agents.

In current Jenkins terminology, controller is preferred over the older term master.

### 5.2 Purpose - Why do we use the controller?

- To manage Jenkins jobs and pipelines.
- To provide the Jenkins web interface.
- To schedule builds and assign work.
- To manage nodes, agents, and credentials.
- To store build history and configuration.
- To coordinate pipeline execution.

### 5.3 Working - How does the controller work?

1. A user creates or starts a Jenkins job.
2. Jenkins evaluates the job configuration and pipeline.
3. The controller determines where the work should run.
4. It schedules tasks and coordinates execution.
5. It tracks the build status and execution results.
6. It presents logs and results in the Jenkins interface.

The controller does not necessarily execute all build commands itself; tasks can be delegated to agents.

### 5.4 Practical Example

You have Jenkins installed as a Windows service on your computer and access it through: `http://localhost:8180`

In this setup, the Jenkins service acts as the controller.

When you run your EKS Terraform pipeline, the controller manages the job and schedules its pipeline steps on the node selected by your Jenkinsfile and configuration.

### 5.5 Interview Questions

#### Q1. What is a Jenkins Controller?

**Answer:** The Jenkins controller is the central component responsible for managing jobs, pipeline scheduling, configuration, agents, and build results.

#### Q2. What are the responsibilities of a controller?

**Answer:** Its responsibilities include job management, pipeline orchestration, agent coordination, configuration management, and providing the web interface.

#### Q3. Should heavy builds run on the controller?

**Answer:** For production environments, it is generally recommended to use dedicated agents for heavy builds and deployment tasks. This helps preserve controller resources and improve workload isolation.

#### Q4. What happens if the Jenkins controller goes down?

**Answer:** Jenkins scheduling and management become unavailable until the controller is restored. Builds running on agents may be affected depending on their connection state, pipeline behavior, and recovery configuration.

#### Q5. How do you secure a Jenkins controller?

**Answer:** Use strong authentication, role-based access control, secure credentials management, restricted network access, regular updates, backups, and least-privilege permissions.

## 6. Jenkins Agent

### 6.1 Definition

A Jenkins agent is a machine or execution environment that connects to the Jenkins controller and runs tasks assigned by it.

An agent may run on Linux, Windows, or another supported environment and can provide specific tools needed by a pipeline.

### 6.2 Purpose - Why do we use agents?

- To execute builds outside the controller.
- To distribute workloads across multiple machines.
- To support different operating systems.
- To provide specific tools and dependencies.
- To improve scalability and workload isolation.
- To run deployment tasks in appropriately configured environments.

### 6.3 Working - How does an agent work?

1. An agent is configured and connected to the controller.
2. Jenkins identifies the agent's labels, availability, and configured executors.
3. A pipeline requests an appropriate agent using its configuration or labels.
4. Jenkins schedules work on a suitable available executor.
5. The agent executes the assigned commands.
6. The execution results and logs are reported back to Jenkins.

The agent must have the necessary software, permissions, connectivity, and resources for the task.

### 6.4 Practical Example

Suppose your Terraform pipeline requires Terraform CLI and AWS authentication. You can configure a Linux EC2 instance as a Jenkins agent with Terraform installed and suitable AWS permissions. A pipeline can then use that agent to execute:

```
terraform init
terraform fmt -check
terraform validate
terraform plan
```

For a production deployment, terraform apply should follow the project's approval and change-control process.

### 6.5 Interview Questions

#### Q1. What is a Jenkins Agent?

**Answer:** A Jenkins agent is a machine or execution environment that runs pipeline tasks assigned by the Jenkins controller.

#### Q2. What is the difference between a controller and an agent?

**Answer:** The controller manages Jenkins and coordinates jobs, while an agent executes assigned build, test, or deployment tasks.

#### Q3. What is an agent label?

**Answer:** An agent label is a name or identifier used to select agents with particular characteristics or capabilities, such as linux, docker, or terraform.

#### Q4. Can Jenkins use multiple agents?

**Answer:** Yes. Jenkins can use multiple agents to distribute workloads, execute tasks on different operating systems, and improve build capacity.

#### Q5. What happens if an agent goes offline?

**Answer:** Jenkins cannot schedule new work on an unavailable agent. Queued tasks may wait for another suitable agent, depending on pipeline requirements and configuration. Running tasks may fail or be interrupted.

#### Q6. What should be installed on a Jenkins agent?

**Answer:** An agent needs the required runtime and connectivity for Jenkins, plus project-specific tools such as Git, Java, Maven, Docker, Terraform, or AWS CLI as needed.

## 7. Jenkins Executor

### 7.1 Definition

An executor is a slot on a Jenkins node that allows Jenkins to run a build or task.

The controller's built-in node and connected agents can each have a configured number of executors.

### 7.2 Purpose - Why do we use executors?

- To control how many tasks run concurrently on a node.
- To manage build capacity.
- To make efficient use of available resources.
- To avoid running more workloads than a machine can safely handle.

### 7.3 Working - How does an executor work?

- If a node has two available executors, it can generally run two eligible tasks simultaneously.
- If both executors are occupied, another task requiring that node must wait until capacity becomes available or another eligible node can run it.
- Actual performance depends on CPU, memory, disk, network, and workload requirements.

### 7.4 Practical Example

Suppose a Linux agent has two executors:

- Executor 1 runs the Terraform validation pipeline.
- Executor 2 runs another project's build.

A third task that requires the same agent waits until a slot becomes available.

Setting more executors does not automatically make the machine faster. Too many concurrent workloads can exhaust system resources.

### 7.5 Interview Questions

#### Q1. What is an executor in Jenkins?

**Answer:** An executor is a slot on a Jenkins node that allows a build or task to run.

#### Q2. What happens when all executors are busy?

**Answer:** Additional tasks requiring that node wait in the queue until an executor becomes available, unless another eligible node can execute them.

#### Q3. Can we increase the number of executors?

**Answer:** Yes, but the number should be based on system capacity and workload requirements.

#### Q4. Is an executor the same as an agent?

**Answer:** No. An agent is the machine or execution environment; an executor is a slot on that node used to run a task.

## 8. End-to-End CI/CD Workflow

This final topic connects all the concepts covered in Part 1.

### 8.1 Definition

An end-to-end CI/CD workflow is the complete automated process that takes a code change from a source repository through validation and delivery to a target environment.

### 8.2 Purpose - Why do we use it?

- To automate the software delivery lifecycle.
- To standardize build and deployment processes.
- To detect problems early.
- To make releases more consistent and traceable.
- To reduce manual intervention.

### 8.3 Working - How does it work?

1. A developer pushes code to GitHub.
2. Jenkins starts the configured pipeline.
3. The controller coordinates the pipeline execution.
4. An appropriate agent checks out the code.
5. The pipeline runs build and test stages.
6. Required quality and security checks run.
7. The pipeline packages the application or validates infrastructure.
8. Deployment proceeds when required checks and approvals pass.
9. Jenkins records the outcome.

### 8.4 Practical Example - Your Terraform and AWS Project

Your EKS Terraform project can follow this workflow:

1. **GitHub:** Terraform source code and Jenkinsfile
2. **Jenkins Controller:** Schedules and coordinates pipeline
3. **Jenkins Agent:** Runs Terraform commands
4. **Terraform Init:** Initializes providers and backend
5. **Terraform Validate & Plan:** Checks configuration and previews changes
6. **Approval:** Reviews proposed infrastructure changes
7. **Terraform Apply:** Creates or updates AWS infrastructure
8. **AWS EKS:** Target infrastructure

**Important:** Terraform apply should only run after the plan has been reviewed and the required approval has been obtained. Use secure AWS credentials or an appropriate IAM role rather than hardcoding access keys.

### 8.5 Interview Questions

#### Q1. Explain your Jenkins CI/CD project.

**Answer:** I have worked on a Jenkins CI/CD pipeline integrated with GitHub, Terraform, and AWS. The pipeline checks out the source code, initializes Terraform, validates the configuration, and generates an execution plan. After reviewing the proposed changes and obtaining the required approval, the pipeline can apply the infrastructure changes to AWS. I also troubleshoot Git checkout, credentials, provider initialization, and pipeline execution issues.

### Q2. How does Jenkins integrate with GitHub?

**Answer:** Jenkins integrates with GitHub through source control configuration and credentials where needed. Webhooks, polling, or manual triggers can start builds when code changes occur.

### Q3. How do you troubleshoot a failed pipeline?

**Answer:** I identify the failed stage, review the Jenkins console output, verify the source code and configuration, check credentials and connectivity, reproduce the issue if necessary, and rerun the pipeline after fixing the root cause.

### Q4. How do you securely deploy infrastructure using Jenkins?

**Answer:** I use secure credentials or IAM roles, least-privilege permissions, protected source control, Terraform plan review, approval gates, and secure remote state management.

### Q5. What is the role of Jenkins in Infrastructure as Code?

**Answer:** Jenkins automates Terraform workflows such as initialization, formatting, validation, planning, approval, and applying infrastructure changes.

## Part 1 - Final Revision

Use this checklist to make sure your fundamentals are complete.

### Study Checklist

- ■ Jenkins definition and purpose
- ■ CI concepts and workflow
- ■ Continuous Delivery vs Continuous Deployment
- ■ Jenkins architecture
- ■ Controller responsibilities
- ■ Agent responsibilities
- ■ Executor and concurrency
- ■ End-to-end CI/CD workflow
- ■ Terraform + AWS project explanation
- ■ Interview questions and answers

### Recommended order for your Jenkins notes

1. Jenkins Introduction
2. Continuous Integration
3. Continuous Delivery and Continuous Deployment
4. Jenkins Architecture
5. Jenkins Controller
6. Jenkins Agent
7. Jenkins Executor
8. End-to-End CI/CD Workflow
9. Interview Questions and Answers

# Jenkins Notes - Part 2: Java Requirements for Jenkins

## 1. Java Requirements for Jenkins

### 1.1 Definition - What is it?

Jenkins is primarily written in Java and requires a supported Java runtime to start and operate. The required Java version depends on the Jenkins release being installed.

Java provides the runtime environment needed to execute the Jenkins application.

### 1.2 Purpose - Why do we need Java?

- To run the Jenkins application.
- To provide the Java Virtual Machine (JVM).
- To support Jenkins services and plugin execution.
- To ensure compatibility with the installed Jenkins version.
- To help maintain a stable and supported Jenkins environment.

### 1.3 Working - How does it work?

1. Install a Java Development Kit (JDK) or suitable Java runtime.
2. Verify that the Java version is supported by your Jenkins release.
3. Ensure the Jenkins service can locate the Java executable.
4. Start the Jenkins service.
5. Jenkins runs on the JVM and initializes its web server and components.

Useful commands on Windows PowerShell:

```
java -version
where.exe java
```

Useful commands on Linux:

```
java -version
which java
```

A JDK includes development tools in addition to the runtime; Jenkins itself does not always require a full JDK for every workload. Build tools such as Maven or Gradle may have their own Java requirements.

### 1.4 Practical Example

Suppose Jenkins is installed on your Windows computer and Java 17 is available at: `C:\Program Files\Java\jdk-17`

You can check the installed Java version:

```
java -version
```

If Jenkins fails to start, check the Jenkins service logs and verify that the configured Java runtime is supported by your Jenkins version.

For your Terraform pipeline, Java runs Jenkins; Terraform and AWS CLI are separate tools that must also be installed or made available wherever the pipeline executes.

### 1.5 Interview Questions

#### Q1. Why does Jenkins require Java?

**Answer:** Jenkins is primarily written in Java and runs on the JVM, so it requires a supported Java runtime.

#### Q2. Which Java version is required for Jenkins?

**Answer:** The required version depends on the Jenkins release. Always check the official system requirements for the specific Jenkins version being installed.

#### Q3. What is the difference between JDK and JRE?

**Answer:** A JDK includes development tools such as the Java compiler along with the runtime components. A traditional JRE provides the components needed to run Java applications. Modern JDK distributions commonly provide the runtime directly.

#### Q4. How do you check the Java version?

**Answer:** On Windows, run `java -version` in PowerShell or Command Prompt. On Linux, run `java -version` in the terminal.

#### Q5. What would you check if Jenkins does not start after a Java upgrade?

**Answer:** I would verify Java compatibility, the Java executable path, service configuration, environment variables where applicable, and Jenkins startup logs.

## 2. Jenkins Installation

### 2.1 Definition - What is it?

Jenkins installation is the process of setting up the Jenkins automation server on a machine so that it can execute jobs and pipelines.

Jenkins can be installed on Windows, Linux, or other supported environments. It can also run in containers or Kubernetes.

### 2.2 Purpose - Why do we install Jenkins?

- To automate software builds and tests.
- To create and manage CI/CD pipelines.
- To integrate source code repositories with automation workflows.
- To automate deployments to development, testing, and production environments.
- To provide a centralized interface for managing automation jobs.

### 2.3 Working - How does Jenkins installation work?

A typical Windows installation involves:

1. Install a Java version supported by the Jenkins release.
2. Download the official Jenkins Windows installer.
3. Run the installer and configure the service settings.
4. Start the Jenkins Windows service.
5. Open the Jenkins web interface.
6. Unlock Jenkins using the initial administrator password.
7. Install the suggested plugins or select the required plugins.
8. Create the first administrator account.
9. Verify that Jenkins is ready to use.

The initial password is generated during first-time setup. The installer or setup page identifies where to find it.

### 2.4 Practical Example

Your Jenkins server is accessible at: `http://localhost:8180`

Because this is a custom port, it is important to check that the Jenkins service is configured to listen on port 8180.

Useful PowerShell commands:

```
Get-Service *jenkins*
```

To check whether a process is listening on port 8180:

```
Get-NetTCPConnection -LocalPort 8180 -State Listen
```

To restart the Jenkins service, if its service name is Jenkins:

```
Restart-Service -Name Jenkins
```

Run PowerShell as Administrator if service permissions require it.

**Important:** These commands are for checking and managing an existing installation. Do not reinstall Jenkins simply because the dashboard is unavailable; first check the service status, port, and logs.

### 2.5 Interview Questions

#### Q1. How do you install Jenkins on Windows?

**Answer:** I install a supported Java version, download and run the official Jenkins installer, configure the service, access the web interface, unlock Jenkins, install the required plugins, and create an administrator account.

#### Q2. What is the default Jenkins port?

**Answer:** Jenkins traditionally uses port 8080 by default. The port can be changed during installation or through the server configuration.

#### Q3. How do you check whether Jenkins is running on Windows?

**Answer:** I use PowerShell's `Get-Service` command to check the Jenkins service status and verify the configured listening port.

#### Q4. What is the Jenkins initial administrator password?

**Answer:** It is a generated secret used to unlock Jenkins during the initial setup process.

#### Q5. What would you do if Jenkins is not accessible in the browser?

**Answer:** I would check the service status, configured port, process logs, port conflicts, and firewall settings before investigating further.

## 3. Jenkins Initial Setup and Dashboard

### 3.1 Definition - What is it?

The Jenkins dashboard is the web-based user interface used to manage jobs, pipelines, build history, agents, plugins, credentials, and system configuration.

Initial setup is the process of unlocking Jenkins and configuring its first administrator account and required components.

### 3.2 Purpose - Why do we use the dashboard?

- To create and manage Jenkins jobs.
- To start, stop, and monitor builds.
- To view pipeline execution and console output.
- To manage plugins, users, and credentials.
- To configure nodes, agents, and system settings.
- To troubleshoot failed builds.

### 3.3 Working - How does it work?

After Jenkins starts:

1. Open the configured Jenkins URL in a browser.
2. Log in with an authorized account.
3. Use the dashboard to create or open a job.
4. Start a build manually or through a configured trigger.
5. Open the build history to inspect results.
6. View the Console Output to troubleshoot failures.
7. Access Manage Jenkins to configure supported system settings.

Dashboard options may vary depending on the installed Jenkins version, permissions, and plugins.

### 3.4 Practical Example

In your environment, open: `http://localhost:8180`

To inspect your Terraform pipeline:

1. Open the relevant Jenkins job.
2. Select Build Now, if the job supports it.
3. Open the new build entry.
4. Select Console Output.
5. Check stages such as Git checkout, Terraform initialization, validation, and planning.

Typical Jenkins build statuses:

| Status | Meaning |
|---|---|
| Success | The build completed successfully. |
| Failure | The build failed or an execution step returned an error. |
| Unstable | The build completed but some configured checks, such as tests, reported problems. |
| Aborted | The build was stopped before normal completion. |

The exact status depends on the pipeline and job configuration.

### 3.5 Interview Questions

**Q1. What is the Jenkins dashboard?**

**Answer:** The Jenkins dashboard is the web interface used to manage jobs, pipelines, build history, plugins, users, and system configuration.

**Q2. What is Console Output?**

**Answer:** Console Output displays the logs produced during a build or pipeline execution and is a primary resource for troubleshooting.

**Q3. How do you manually trigger a Jenkins job?**

**Answer:** Open the job and select Build Now or the corresponding build action available for that job.

**Q4. What is the purpose of Manage Jenkins?**

**Answer:** It provides administrative settings for Jenkins, such as plugins, system configuration, security, nodes, and other supported management features.

**Q5. How do you troubleshoot a failed build from the dashboard?**

**Answer:** I open the failed build, inspect Console Output, identify the failing stage or command, verify the relevant configuration and dependencies, and then fix and rerun the pipeline.

## 4. Jenkins Plugins

### 4.1 Definition - What is it?

A Jenkins plugin is an extension that adds functionality or integrates Jenkins with external tools and services.

Jenkins provides a core set of capabilities, while plugins extend it to support source control, build tools, cloud platforms, notifications, pipeline features, and other integrations.

### 4.2 Purpose - Why do we use plugins?

- To integrate Jenkins with GitHub and other source control systems.
- To support different build and deployment tools.
- To connect Jenkins to cloud platforms.
- To enable additional pipeline features.
- To publish test reports and build artifacts.
- To send notifications about pipeline results.
- To manage authentication and credentials integrations.

### 4.3 Working - How do plugins work?

1. An administrator opens the Jenkins plugin management interface.
2. The administrator searches for the required plugin.
3. The plugin is installed or updated, subject to compatibility and permissions.
4. Jenkins loads the plugin and exposes its functionality.
5. Jobs or pipelines use the plugin's features.

Some plugin changes require Jenkins to restart. Plugin dependencies and compatibility should be checked before updating production installations.

### 4.4 Practical Example

For a Jenkins + Terraform + AWS project, the following plugins may be useful:

| Plugin | Purpose |
|---|---|
| Pipeline | Defines and executes pipeline jobs. |
| Git | Checks out source code from Git repositories. |
| GitHub integration | Supports GitHub-related integrations and triggers. |
| Credentials Binding | Exposes configured credentials securely to supported build steps. |
| AWS Credentials | Provides AWS credential integration for compatible pipelines and plugins. |
| Docker-related plugins | Provide additional Docker integrations where needed. |

Plugin names and exact capabilities can vary by release. Not every pipeline needs every plugin; standard Pipeline and Git features are often sufficient for basic workflows.

**Best practice:** Install only the plugins you need, keep them updated after compatibility checks, and remove unused plugins where appropriate.

### 4.5 Interview Questions

#### Q1. What is a Jenkins plugin?

**Answer:** A Jenkins plugin extends Jenkins functionality or integrates it with an external tool or service.

#### Q2. Why is the Git plugin required?

**Answer:** The Git plugin enables Jenkins jobs to retrieve source code from Git repositories.

#### Q3. What is the Credentials Binding plugin used for?

**Answer:** It allows supported build steps to access configured credentials through controlled environment variables or other supported bindings.

#### Q4. Can too many plugins cause problems?

**Answer:** Yes. Unnecessary or incompatible plugins can increase maintenance overhead, introduce security risks, create dependency conflicts, and affect stability.

#### Q5. How do you update Jenkins plugins safely?

**Answer:** I review compatibility and dependencies, back up Jenkins configuration, test updates in a non-production environment when possible, and apply updates according to the maintenance plan.

## 5. Jenkins Users and Authentication

### 5.1 Definition - What is it?

A Jenkins user is an identity used to access Jenkins. Authentication verifies who the user is, while authorization determines which actions that user is allowed to perform.

Jenkins supports its own user database and can integrate with external identity providers, depending on configuration and plugins.

## 5.2 Purpose - Why do we manage users?

- To control access to Jenkins.
- To separate administrator and developer responsibilities.
- To prevent unauthorized changes to pipelines.
- To protect credentials and sensitive configuration.
- To provide accountability for administrative and operational actions.
- To enforce least-privilege access.

## 5.3 Working - How does it work?

1. An administrator configures the security realm, which defines how users authenticate.
2. Users sign in using the configured authentication method.
3. Jenkins evaluates the user's permissions.
4. Users can access only the features permitted by the active authorization strategy.
5. Administrators periodically review and update access as responsibilities change.

Authentication and authorization are different:

- **Authentication:** Who are you?
- **Authorization:** What are you allowed to do?

## 5.4 Practical Example

Consider a team with three roles:

| Role | Typical responsibilities |
|---|---|
| Jenkins Administrator | Manages system configuration, plugins, security, and credentials. |
| Developer | Views and runs permitted jobs and reviews build logs. |
| Release Engineer | Manages or executes approved deployment workflows. |

The exact permissions should be configured using the authorization mechanism available in your Jenkins installation.

For example, developers should not automatically receive permission to change global credentials or modify production deployment settings.

## 5.5 Interview Questions

### Q1. What is authentication in Jenkins?

**Answer:** Authentication verifies the identity of a user attempting to access Jenkins.

### Q2. What is authorization in Jenkins?

**Answer:** Authorization determines which Jenkins actions and resources an authenticated user can access.

### Q3. What is the difference between an administrator and a developer?

**Answer:** An administrator manages Jenkins configuration and security, while a developer generally works with jobs and pipelines within the permissions granted to them.

### Q4. How do you secure Jenkins user access?

**Answer:** I use strong authentication, role-based or matrix-based permissions where appropriate, least privilege, restricted administrative access, and periodic access reviews.

### Q5. Why should developers not have unrestricted administrator access?

**Answer:** Unrestricted access can allow accidental or unauthorized changes to security settings, credentials, jobs, and production deployments.

# 6. Jenkins Credentials and Secret Management

## 6.1 Definition - What is it?

Jenkins Credentials is a built-in mechanism for storing and managing sensitive information required by jobs and pipelines.

Credentials can include usernames and passwords, secret text, SSH private keys, and username/password combinations. Additional credential types may be supported through plugins.

Secrets should be stored in Jenkins' credential store or an appropriate external secret-management system, rather than hardcoded in pipeline files.

## 6.2 Purpose - Why do we use credentials?

- To authenticate Jenkins with GitHub.
- To access private Git repositories.
- To authenticate with AWS or other cloud platforms.
- To connect to remote servers using SSH.
- To access container registries.
- To avoid exposing passwords, tokens, and access keys in source code.
- To control access to sensitive values.

## 6.3 Working - How do Jenkins credentials work?

1. An authorized administrator creates a credential in Jenkins.
2. The credential is assigned an ID and an appropriate scope.
3. A pipeline references the credential by its ID.
4. Jenkins makes the credential available to a supported step or plugin.
5. The pipeline uses the credential to authenticate with the required service.

Jenkins attempts to mask supported secret values in logs, but masking is not a complete security boundary.

Important security practices:

- Never hardcode secrets in a Jenkinsfile.
- Never commit passwords, tokens, or private keys to GitHub.
- Restrict which jobs and users can access credentials.
- Avoid printing secrets in logs.
- Prefer short-lived credentials or IAM roles when supported.
- Rotate credentials if they are exposed.

## 6.4 Practical Example

Suppose your Jenkins pipeline must authenticate with AWS to execute Terraform commands. You have a Jenkins credential ID named `aws-jenkins-credentials`.

A pipeline may use the Credentials Binding plugin for static AWS credentials, for example:

```
withCredentials([
    [$class: 'AmazonWebServicesCredentialsBinding',
     credentialsId: 'aws-jenkins-credentials']
]) {
    bat 'aws sts get-caller-identity'
}
```

This is an illustrative example for a compatible installation with the required AWS credentials integration.

The command verifies which AWS identity the CLI is using. It does not create or modify AWS resources.

**For production:** Prefer an IAM role or another short-lived credential mechanism when possible. The credential type and binding syntax depend on your Jenkins plugins and execution environment.

A GitHub Personal Access Token should also be stored in Jenkins Credentials if the pipeline needs it for private repository access. Never paste the token directly into the Jenkinsfile.

### 6.5 Interview Questions

**Q1. What are Jenkins Credentials?**

**Answer:** Jenkins Credentials is a mechanism for securely storing and accessing sensitive authentication information required by jobs and pipelines.

**Q2. What types of credentials does Jenkins support?**

**Answer:** Common types include username/password, secret text, secret file, and SSH private key credentials. Other types may be available through plugins.

**Q3. How do you use credentials in a Jenkins pipeline?**

**Answer:** I create a credential with an appropriate ID and scope, then reference it through a supported pipeline step or plugin rather than hardcoding the secret.

**Q4. How do you integrate Jenkins with AWS securely?**

**Answer:** I prefer IAM roles or short-lived credentials where possible. Otherwise, I store AWS credentials in Jenkins Credentials, restrict access, and avoid exposing secrets in source code or logs.

**Q5. Are Jenkins credentials completely safe from exposure?**

**Answer:** Jenkins provides encrypted credential storage and mechanisms to restrict access and mask secrets, but credentials can still be exposed through insecure pipeline code, excessive permissions, compromised agents, or malicious build steps. Proper access controls and secret-handling practices are essential.

**Q6. What is the difference between a credential ID and a credential value?**

**Answer:** The credential ID is the identifier used by a pipeline to reference a stored credential. The credential value is the sensitive information itself, such as a token or password.

## 7. End-to-End Practical Revision

Now connect all six topics to a real Jenkins setup.

1. **Java Runtime:** Provides the runtime for Jenkins
2. **Jenkins Installation:** Installs and starts the service
3. **Jenkins Dashboard:** Provides the management interface
4. **Plugins:** Integrate GitHub, AWS and pipeline features
5. **Users & Permissions:** Control who can access and manage Jenkins
6. **Credentials:** Provide controlled access to sensitive values
7. **CI/CD Pipeline:** Executes the configured automation

## 8. Real-Time Scenario Interview Questions

**Scenario 1: Jenkins is not starting after installation. What do you check?**

**Answer:**

- Check whether the Jenkins service is running.
- Verify Java compatibility and the configured Java path.
- Review Jenkins startup logs.
- Check for port conflicts.
- Verify service permissions and configuration.
- Apply the fix and restart the service if required.

**Scenario 2: Jenkins cannot access a private GitHub repository. What do you check?**

**Answer:**

- Verify the repository URL and branch.
- Confirm the configured Git credentials.
- Check token permissions and expiration.
- Verify Git installation and network connectivity.

- Review the Jenkins console output.
- Test checkout again after correcting the issue.

### Scenario 3: AWS commands fail inside Jenkins, but work on your local terminal. What do you check?

**Answer:**

- Verify which machine or agent executes the pipeline.
- Check the AWS CLI installation on that execution node.
- Verify the AWS identity using `aws sts get-caller-identity`.
- Check the credential binding or IAM role configuration.
- Verify region settings and IAM permissions.
- Review the logs without exposing secrets.

### Scenario 4: A developer can run a build but cannot modify a credential. Is this expected?

**Answer:** Yes. Jenkins permissions can allow users to run permitted jobs without giving them administrative access to modify global credentials or security settings.